Featured examples

- Train a miniGPT language model with JAX AI Stack
- LoRA/QLoRA finetuning for LLM using Tunix
- Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA
- Loading Hugging Face Transformers Checkpoints
- 8-bit Integer Quantization in Keras
- Float8 training and inference with a simple Transformer model
- Pretraining a Transformer from scratch with KerasHub
- Simple MNIST convnet
- Image classification from scratch using Keras 3
- Image Classification with KerasHub

```
!pip install pyspark kagglehub pandas matplotlib
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/
Requirement already satisfied: kagglehub in /usr/local/lib/python3.1
Requirement already satisfied: pandas in /usr/local/lib/python3.12/d
Requirement already satisfied: matplotlib in /usr/local/lib/python3.
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/loc
Requirement already satisfied: kagglesdk<1.0,>=0.1.14 in /usr/local/
Requirement already satisfied: packaging in /usr/local/lib/python3.1
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/d
Requirement already satisfied: requests in /usr/local/lib/python3.12
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dis
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/pythc
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/pythor
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pyth
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/py
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/pythor
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/p
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/p
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.1
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/py
Requirement already satisfied: protobuf in /usr/local/lib/python3.12
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/pythor
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
```

```
import kagglehub
import pandas as pd
import time
import matplotlib.pyplot as plt

from pyspark.sql import SparkSession
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("mkechinov/ecommerce-behavior-dat

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'ecommerce-behavior-data-
Path to dataset files: /kaggle/input/ecommerce-behavior-data-from-mu
```

```
import kagglehub
import os
import pandas as pd

# download dataset
path = kagglehub.dataset_download("mkechinov/ecommerce-behavior-dat

# list files
files = os.listdir(path)

# load file
file_path = path + "/" + files[0]

df = pd.read_csv(file_path, nrows=100000)

df.head()
```

```
Using Colab cache for faster access to the 'ecommerce-behavior-data-
```

	event_time	event_type	product_id	category_id	ca
0	2019-11-01 00:00:00 UTC	view	1003461	2053013555631882655	electroni
1	2019-11-01 00:00:00 UTC	view	5000088	2053013566100866035	appliances.se
2	2019-11-01 00:00:01 UTC	view	17302664	2053013553853497655	
3	2019-11-01 00:00:01 UTC	view	3601530	2053013563810775923	appliances.k
4	2019-11-01 00:00:01 UTC	view	1004775	2053013555631882655	electroni

Next steps: Generate code with df | New interactive sheet

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   event_time     100000 non-null  object 
 1   event_type     100000 non-null  object 
 2   product_id     100000 non-null  int64  
 3   category_id    100000 non-null  int64  
 4   category_code  66160 non-null   object 
 5   brand          84224 non-null   object 
 6   price          100000 non-null  float64
 7   user_id        100000 non-null  int64  
 8   user_session   100000 non-null  object 
dtypes: float64(1), int64(3), object(5)
memory usage: 6.9+ MB
```

	0
event_time	0
event_type	0
product_id	0
category_id	0
category_code	33840
brand	15776
price	0
user_id	0
user_session	0

dtype: int64

```
# fill missing values
df['category_code'] = df['category_code'].fillna("unknown")
df['brand'] = df['brand'].fillna("unknown")

# convert time column
df['event_time'] = pd.to_datetime(df['event_time'])

df.head()
```

	event_time	event_type	product_id	category_id	c
0	2019-11-01 00:00:00+00:00	view	1003461	2053013555631882655	electro
1	2019-11-01 00:00:00+00:00	view	5000088	2053013566100866035	appliances.s
2	2019-11-01 00:00:01+00:00	view	17302664	2053013553853497655	
3	2019-11-01 00:00:01+00:00	view	3601530	2053013563810775923	appliance
4	2019-11-01 00:00:01+00:00	view	1004775	2053013555631882655	electro

Next steps: Generate code with df | New interactive sheet

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/loc
```

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("EcommerceRealtimeAnalytics") \
    .getOrCreate()
```

```
spark_df = spark.createDataFrame(df)

spark_df.show(5)
```

```
+-------------------+----------+----------+-------------------+-----
|         event_time|event_type|product_id|        category_id|
+-------------------+----------+----------+-------------------+-----
|2019-11-01 00:00:00|      view|   1003461|2053013555631882655|elect
|2019-11-01 00:00:00|      view|   5000088|2053013566100866035|appli
|2019-11-01 00:00:01|      view|  17302664|2053013553853497655|
|2019-11-01 00:00:01|      view|   3601530|2053013563810775923|appli
|2019-11-01 00:00:01|      view|   1004775|2053013555631882655|elect
+-------------------+----------+----------+-------------------+-----
only showing top 5 rows
```

```
spark_df.groupBy("event_type").count().show()
```

```
+----------+-----+
|event_type|count|
+----------+-----+
|  purchase| 1422|
|      view|97489|
|      cart| 1089|
+----------+-----+
```

```
spark_df.groupBy("product_id") \
    .count() \
    .orderBy("count", ascending=False) \
    .show(10)
```

```
+----------+-----+
|product_id|count|
+----------+-----+
|   1004856| 1132|
|   1004767|  942|
|   1005115|  906|
|   1004833|  566|
|   1004249|  527|
|   1005105|  526|
|   1004870|  517|
|   4804056|  495|
|   1005160|  489|
|   1002544|  470|
+----------+-----+
only showing top 10 rows
```

```
spark_df.groupBy("user_id") \
    .count() \
    .orderBy("count", ascending=False) \
    .show(10)
```

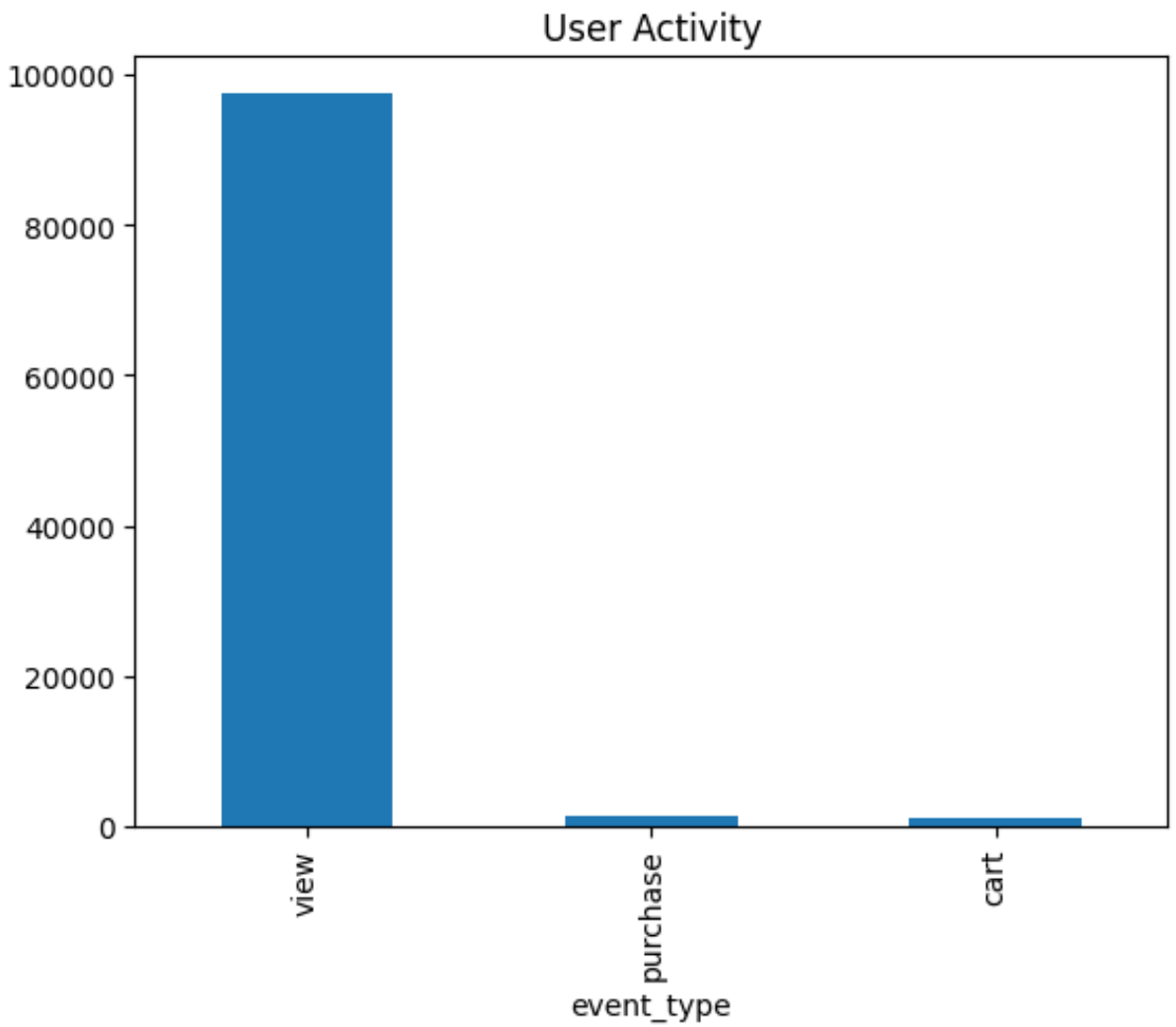
```
+---------+-----+
|  user_id|count|
+---------+-----+
|513002539|  140|
|534949460|   99|
|514127132|   97|
|530337676|   93|
|534838878|   87|
|515129681|   86|
|566301046|   86|
|512483076|   85|
|557212976|   83|
|563911925|   82|
+---------+-----+
only showing top 10 rows
```

```
import time

chunks = [df[i:i+5000] for i in range(0, len(df), 5000)]

for i, chunk in enumerate(chunks[:5]):
    print(f"\n--- Real-Time Batch {i+1} ---\n")

    spark_chunk = spark.createDataFrame(chunk)

    spark_chunk.groupBy("event_type").count().show()

    time.sleep(2)
```

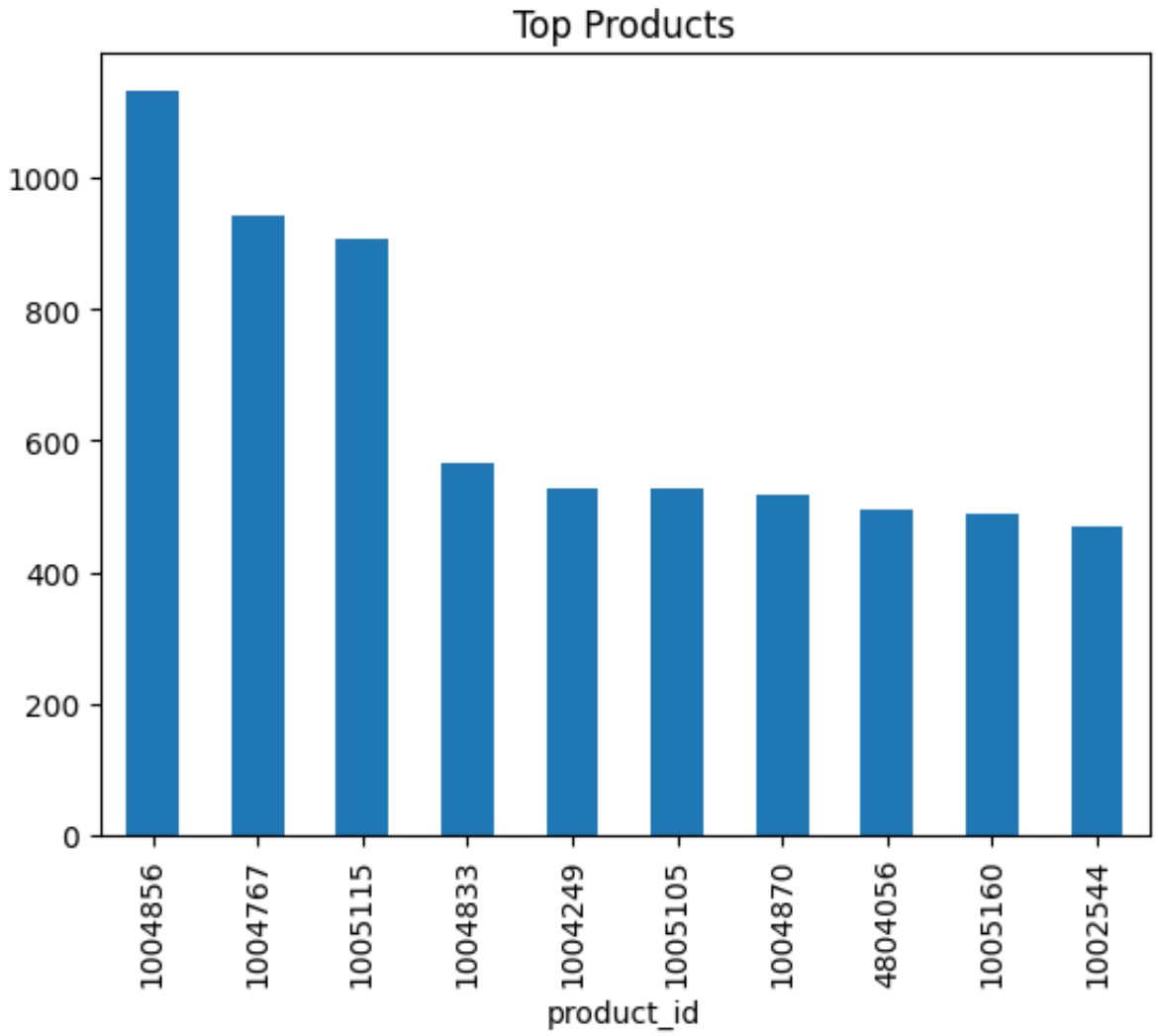
```

--- Real-Time Batch 1 ---

+----------+-----+
|event_type|count|
+----------+-----+
|  purchase|   54|
|      view| 4894|
|      cart|   52|
+----------+-----+


--- Real-Time Batch 2 ---

+----------+-----+
|event_type|count|
```

```
+----------+-----+
|  purchase|   48|
|      view| 4911|
|      cart|   41|
+----------+-----+


--- Real-Time Batch 3 ---

+----------+-----+
|event_type|count|
+----------+-----+
|  purchase|   47|
|      view| 4923|
|      cart|   30|
+----------+-----+


--- Real-Time Batch 4 ---

+----------+-----+
|event_type|count|
+----------+-----+
|  purchase|   34|
|      view| 4943|
|      cart|   23|
+----------+-----+


--- Real-Time Batch 5 ---

+----------+-----+
|event_type|count|
+----------+-----+
|  purchase|   53|
|      view| 4892|
|      cart|   55|
+----------+-----+
```

```
import matplotlib.pyplot as plt

event_counts = df["event_type"].value_counts()

event_counts.plot(kind='bar')
plt.title("User Activity")
plt.show()
```



```
top_products = df["product_id"].value_counts().head(10)

top_products.plot(kind='bar')
plt.title("Top Products")
plt.show()
```



```
import pandas as pd

transactions = df[['user_id', 'product_id', 'price']].copy()
transactions['purchase'] = 1

transactions.head()
```

	user_id	product_id	price	purchase
0	520088904	1003461	489.07	1
1	530496790	5000088	293.65	1
2	561587266	17302664	28.31	1
3	518085591	3601530	712.87	1
4	558856683	1004775	183.27	1

Next steps: Generate code with transactions | New interactive sheet

```
from pyspark.sql.functions import col, avg
import pandas as pd

# Assume 'reviews' dataframe should also come from 'df' but with a
# Since 'df' does not have a 'rating' column, we will create a dumm
reviews = df[['user_id', 'product_id']].copy()
# Add a dummy 'rating' column. For simplicity, let's assign a const
reviews['rating'] = 4.0 # Placeholder rating

# ---- Rename columns to avoid ambiguity ----
spark_transactions = spark.createDataFrame(transactions) \
    .withColumnRenamed("price", "transaction_price")

spark_reviews = spark.createDataFrame(reviews)

# ---- Join Data ----
customer_view = spark_df.join(
    spark_transactions,
    on=['user_id', 'product_id'],
    how='left'
).join(
    spark_reviews,
    on=['user_id', 'product_id'],
    how='left'
)
```

```
print("Unified Customer View:")
customer_view.show(5)

# ---- Customer Insights (FIXED) ----
customer_view.groupBy("user_id") \
    .agg(
        avg("transaction_price").alias("avg_spent"),
        avg("rating").alias("avg_rating")
    ).show(10)
```

```
Unified Customer View:
+---------+----------+-------------------+----------+--------------
|  user_id|product_id|         event_time|event_type|        categor
+---------+----------+-------------------+----------+--------------
|518085591|   3601530|2019-11-01 00:00:01|      view|205301356381077
|520088904|   1003461|2019-11-01 00:00:00|      view|205301355563188
|520088904|   1003461|2019-11-01 00:00:00|      view|205301355563188
|520088904|   1003461|2019-11-01 00:00:00|      view|205301355563188
|520088904|   1003461|2019-11-01 00:00:00|      view|205301355563188
+---------+----------+-------------------+----------+--------------
only showing top 5 rows
+---------+------------------+----------+
|  user_id|         avg_spent|avg_rating|
+---------+------------------+----------+
|514096409|192.83999999999997|       4.0|
|522037659|283.65839080459654|       4.0|
|516276619|326.89205882352934|       4.0|
|564730776| 167.2262820512828|       4.0|
|534722911|           337.694|       4.0|
|561502959|241.55458333333334|       4.0|
|533308184|160.40708978328215|       4.0|
|552098781|            273.37|       4.0|
|512687338|2373.3017187499963|       4.0|
|517647424|1337.2299999999998|       4.0|
+---------+------------------+----------+
only showing top 10 rows
```

```
from pyspark.sql.functions import expr

# ================================
# STEP 13: BASIC SECURITY
# ================================

print("Applying Security Layer...")

# ---- 1. Data Masking ----
# Mask user_id (simulate privacy protection)
customer_view_secured = customer_view.withColumn(
    "user_id_masked",
    expr("user_id % 1000")
```

```
)

print("Masked User IDs:")
customer_view_secured.select("user_id", "user_id_masked").show(5)


# ---- 2. Role-Based Access (Remove Sensitive Data) ----
# Remove original user_id (simulate restricted access)
secure_view = customer_view_secured.drop("user_id")

print("Secure View (Sensitive Data Removed):")
secure_view.show(5)


# ---- 3. Optional: Show Only Limited Columns (Dashboard View) ----
dashboard_view = secure_view.select(
    "product_id",
    "event_type",
    "transaction_price",
    "rating",
    "user_id_masked"
)

print("Dashboard View (Safe Data for Analytics):")
dashboard_view.show(5)
```

```
Applying Security Layer...
Masked User IDs:
+---------+--------------+
|  user_id|user_id_masked|
+---------+--------------+
|518085591|           591|
|520088904|           904|
|520088904|           904|
|520088904|           904|
|520088904|           904|
+---------+--------------+
only showing top 5 rows
Secure View (Sensitive Data Removed):
+----------+-------------------+----------+-------------------+-----
|product_id|         event_time|event_type|        category_id|
+----------+-------------------+----------+-------------------+-----
|   3601530|2019-11-01 00:00:01|      view|2053013563810775923|appli
|   1004775|2019-11-01 00:00:01|      view|2053013555631882655|elect
|   1004775|2019-11-01 00:00:01|      view|2053013555631882655|elect
|   1004775|2019-11-01 00:00:01|      view|2053013555631882655|elect
|   1004775|2019-11-01 00:00:01|      view|2053013555631882655|elect
+----------+-------------------+----------+-------------------+-----
only showing top 5 rows
Dashboard View (Safe Data for Analytics):
```

```
+----------+----------+-----------------+------+--------------+
|product_id|event_type|transaction_price|rating|user_id_masked|
+----------+----------+-----------------+------+--------------+
|   3601530|      view|           712.87|   4.0|           591|
|   1004775|      view|           183.27|   4.0|           683|
|   1004775|      view|           183.27|   4.0|           683|
|   1004775|      view|           183.27|   4.0|           683|
|   1004775|      view|           183.27|   4.0|           683|
+----------+----------+-----------------+------+--------------+
only showing top 5 rows
```

Start coding or generate with AI.